ALTINBAŞ UNIVERSITY

Faculty of Engineering and Architecture
Software Engineering Department

GRADUATION PROJECT

WEEK 3
ARCHITECTURE AND TECHNICAL DESIGN

Project: Pharmacy Management System

Student: Deya Ghanem
Academic year: 2026–2027

Repository: github.com/deya-ghanem/pharmacy-management-system

Design-stage report based on the Week 2 requirements. This document describes a proposed architecture; it does not claim that the full application has been implemented.

1. Requirements-to-Design Traceability

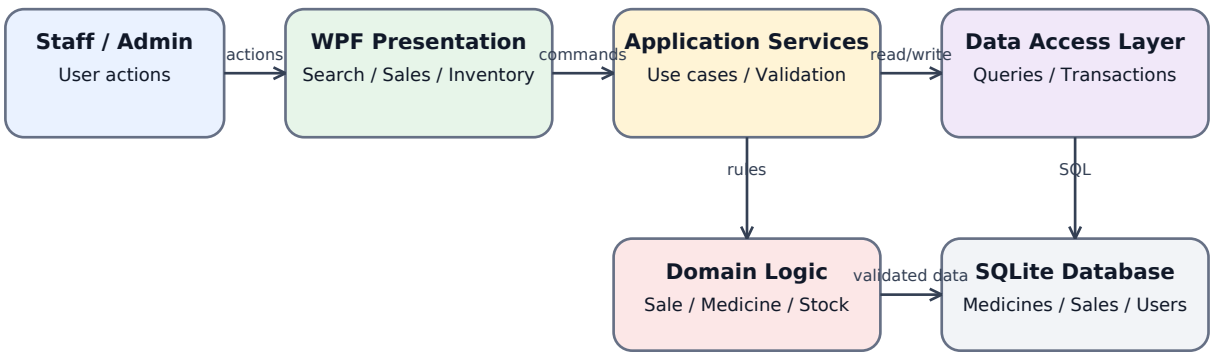
The following requirements are mapped to the proposed components and verification approach.

ID	Requirement	Design response	Verification
FR-01	Search by name or barcode	WPF search calls an application service; the data-access layer queries matching fields in SQLite.	Seed records and verify name/barcode matches and no-match behavior.
FR-03	Add items and calculate total	Sales UI collects sale lines; domain logic validates quantities and calculates line and sale totals.	Test multiple items, quantity boundaries, and total calculations.
FR-04	Save sale and update stock	Sale header, sale lines, and stock deductions are handled in one SQLite transaction; failures roll back.	Test successful sale, insufficient stock, and forced database failure.
NFR-01	Search within 2 seconds with 5,000 records	Index suitable search fields where useful; keep query work in data access.	Seed 5,000 records and time ten searches; report the observed results.
NFR-02	Sale and inventory consistency	Transaction boundaries and service-level validation prevent partial sale/stock updates.	Test success, cancellation, insufficient stock, database error, and repeat confirmation.

2. Proposed Architecture

The initial release is a Windows desktop application intended for one pharmacy computer. It uses four logical layers: presentation, application services, domain logic, and data access. SQLite stores data locally. External APIs, cloud dependencies, and multi-branch synchronization are outside the initial scope.

Architecture diagram



Deployment: one Windows PC; all database access goes through the data-access layer.

Figure 3.1. Staff interact with the WPF presentation layer, which calls application services. Services coordinate validation and domain rules and use the data-access layer for persistence. Only the data-access layer communicates directly with SQLite. This separation supports maintainability and reduces the risk of UI code bypassing validation or transaction handling.

3. Component Responsibilities

Component	Responsibility	Input → output
WPF Presentation	Displays search, sales, and inventory screens; gathers user actions and shows validation feedback.	Search text / sale lines → results, totals, confirmations, errors.
Application Services	Coordinates use cases and calls domain and data-access components.	Use-case request → result or error.
Domain Logic	Validates sale quantities and stock rules; calculates totals.	Sale request → validated sale and calculated total.
Data Access Layer	Encapsulates SQL queries, persistence, and transaction handling.	Queries/entities → database results or persistence status.
SQLite Database	Persists medicine, sale, sale-item, and user records.	Stored records and transaction outcomes.

4. End-to-End Sale Flow

1. Staff searches for a medicine by name or barcode.

2. The UI sends the query to an application service; data access queries SQLite.

3. Matching medicines and available quantities are returned to the UI.

4. Staff selects a medicine and quantity; domain logic validates stock and calculates the total.

5. On confirmation, one database transaction inserts the sale header and sale lines and decrements stock.

6. If the transaction commits, the UI displays confirmation. If a validation or database error occurs, the operation fails, the transaction is rolled back where applicable, and the UI shows an error. Stock must remain unchanged after a failed sale.

Important implementation rule: do not display a successful receipt until the database commit has succeeded.

5. Preliminary Data Dictionary

Entity	Key fields	Rules / notes
Medicine	MedicineId, Name, Barcode, Strength, Form, UnitPrice, QuantityInStock, ExpiryDate, IsActive	Name is required; stock must be non-negative; barcode should be unique when present.
Sale	SaleId, CreatedAt, UserId, TotalAmount, Status	UserId identifies the staff member who recorded the sale.
SaleItem	SaleItemId, SaleId, MedicineId, Quantity, UnitPriceAtSale, LineTotal	Quantity must be greater than zero; preserve the unit price used at sale time.
User	UserId, Username, PasswordHash, Role, IsActive	Username is unique; never store plaintext passwords.

Relationships: each SaleItem belongs to one Sale and references one Medicine. A Sale may contain multiple SaleItems. User identifies the staff member associated with a sale.

Currency, tax, and discount rules were not specified in Week 2 and must be confirmed before implementation. This is a preliminary entity sketch, not a final database schema.

6. Technology Decisions

ADR-01 — C# + WPF on modern .NET

Alternatives: Windows Forms or Python/PySide6. Rationale: WPF is Windows-focused and supports XAML, layout controls, and data binding, which suit the initial Windows desktop target and a multi-screen application. Trade-off: the UI is not cross-platform.

Reference: Microsoft Learn — WPF overview

ADR-02 — SQLite for local storage

Alternatives: PostgreSQL or SQL Server. Rationale: SQLite supports transactions and avoids a separate database server for a single-PC prototype. Trade-off: if multi-branch or high-concurrency requirements emerge, the storage design must be reassessed.

Reference: SQLite — SQLite Is Transactional

These are proposed technical choices, not instructor-imposed constraints. The layered monolithic desktop design avoids unnecessary server complexity in the first version.

7. Security, Failure Handling, and Risks

Require authentication for privileged actions and role-based authorization for edits. Validate required fields, positive sale quantities, and non-negative stock. Store password hashes rather than plaintext passwords. If a database error occurs during a sale, roll back the transaction and show a clear error. Backup and restore must be tested before real use.

Risk	Impact	Mitigation
Partial sale persistence or duplicate stock deduction	Incorrect sales records or inventory quantities.	Use one transaction; test forced failure, cancellation, insufficient stock, and repeated confirmation.
Slow or ambiguous medicine search	Checkout delay or selection of the wrong medicine.	Index appropriate fields, validate barcode uniqueness where present, and test 5,000 records against the 2-second target.

8. Week 4 Prototype Acceptance Goal

Demonstrate a Windows desktop screen where a user searches seeded medicine records by name or barcode, selects a medicine, enters a quantity, confirms a test sale, and sees a saved sale with reduced stock. Evidence should include a UI screenshot and database output before and after the sale. If full integration takes longer, first prove the transaction behavior through a minimal UI or test harness.

9. Files and Next Actions

Add the architecture diagram and its editable source, component interfaces, technology decisions, sale-flow source, and Week 3 progress log to GitHub. Update the README with an architecture summary and links to the design artifacts. Update the cumulative graduation report and push meaningful commits. Add actual commit URLs after the files have been uploaded; do not claim repository evidence that is not yet present.

10. Progress Log

Completed in this design draft: proposed layered architecture; mapped three functional and two non-functional requirements; described component responsibilities and interfaces; documented sale flow and failure handling; sketched preliminary data entities; recorded two technology decisions and two risks.

Next: confirm requirements that remain unspecified, add the design artifacts to GitHub, then implement and test the Week 4 vertical slice.

References

1. Microsoft Learn, WPF overview: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/index
2. SQLite, SQLite Is Transactional: https://www.sqlite.org/transactional.html
3. Microsoft .NET, releases and support: https://learn.microsoft.com/en-us/dotnet/core/releases-and-support